

Vibe Bot — Onboarding

Документ для нового участника проекта. Читай по порядку. Двойная аудитория:

- **Часть А (Запуск).** Минимум, чтобы поднять бота локально и/или развернуть на проде. Можно делать «по копипасте».
- **Часть Б (Код и архитектура).** Куда смотреть, что куда писать, типичные ловушки. Для разработчика, который будет править фиши.

Сначала прочитай `__` — это «единственный источник правды» по правилам. Этот файл — практическое приложение к нему.

0. TL;DR за 30 секунд

- **Что:** Telegram-бот знакомств для anime/manga/art/creative аудитории. Подбор по фанdomам/интересам/вайбу. Premium через Telegram Payments. Лента «Аллея креаторов» как Mini App. Полное ТЗ — `docs/TZ_v1.1.docx`.
- **Стек:** Python 3.12 · aiogram 3 (async) · SQLAlchemy 2 (async) · PostgreSQL 16 · Redis 7 · FastAPI · Vite/React 18 (Mini App) · Docker compose · ruff · pytest · uv.
- **Прод-инсталляция:** Linux-сервер, всё в `docker compose`. Бот — long-polling контейнер `vibe-bot`. API для Mini App — `vibe-api` под uvicorn. nginx на хосте отдаёт TLS и проксирует Mini App.
- **Где править:** см. [§7 «Карта кода»](#). Не угадывай куда — карта в этом файле и в .

Часть А — Запуск

1. Что должно стоять на компьютере (локалка)

Что	Зачем	Как поставить
Docker + docker compose v2	вся БД/Redis/бот крутятся в контейнерах	<code>sudo apt install docker.io docker-compose-plugin</code> (Ubuntu) или Docker Desktop (Win/Mac)
Python 3.12	для запуска тестов / линтера локально вне контейнера	<code>pyenv install 3.12.x</code> или системный
uv	пакетный менеджер, заменяет pip+poetry	<code>curl -LsSf https://astral.sh/uv/install.sh sh</code>
git	очевидно	системный

He Windows-friendly. Полная поддержка только под Linux/macOS. На Windows используй WSL2.

После установки добавь `~/.local/bin` в `PATH` — там лежит `uv`:

```
echo 'export PATH="$HOME/.local/bin:$PATH"' >> ~/.bashrc && source ~/.bashrc
```

2. Локальный запуск (5 минут)

```
git clone <repo-url> vibe-bot
cd vibe-bot

# 1. Конфиг — скопирую пример и впиши свой BOT_TOKEN.
cp .env.example .env
nano .env          # минимум: BOT_TOKEN=<токен от @BotFather>
                  # для локалки: ADMIN_TELEGRAM_IDS=<твой Telegram id через @userinfobot>

# 2. Зависимости в виртуалку.
uv sync

# 3. Поднимаем БД + Redis (без бота — его запустим polling-режимом отдельно).
docker compose -f docker/docker-compose.yml up -d postgres redis

# 4. Накатываем миграции на пустую БД.
uv run alembic upgrade head

# 5. Запускаем бот long-polling (для разработки/демки).
uv run python -m app.main --polling
```

Открой Telegram → найди своего бота по username → `/start` . Должно работать.

Если бот не отвечает / падает: см. [§9 «Troubleshooting»](#).

2а. Локальный запуск внутри Docker (как в проде)

Если хочется проверить ровно ту же сборку, что и на сервере:

```
docker compose -f docker/docker-compose.yml up --build
```

Поднимутся четыре контейнера: `vibe-postgres` , `vibe-redis` , `vibe-bot` (polling), `vibe-api` (FastAPI для Mini App). Логи: `docker compose -f docker/docker-compose.yml logs -f bot` .

Важно: при запуске бота **вне** docker (вариант из §2 п.5) переопредели `DATABASE_URL` и `REDIS_URL` на `localhost` , иначе бот будет искать `postgres` / `redis` в DNS контейнерной сети и упадёт с `gaierror: Temporary failure in name resolution` . Пример: см. `/tmp/run_bot.sh` или просто экспорти в шелле перед запуском: `bash export DATABASE_URL=postgresql+asyncrep://vibe:vibe@localhost:5432/vibe export REDIS_URL=redis://localhost:6379/0`

3. Production — деплой на сервер

3.1. Предусловия на сервере

- Linux (Ubuntu 22.04+ или аналог), пользователь с правами `sudo` .
- Установлены `docker` , `docker compose v2` , `nginx` , `certbot` (для TLS).
- Открыты порты: 22 (SSH), 80/443 (HTTPS-вебхук, Mini App).
- Доменное имя с A-записью на сервер. У нас сейчас — `forbot.credentialn8n.ru` .

3.2. Первый деплой

```
# На сервере
sudo mkdir -p /opt/vibe-bot && sudo chown $USER /opt/vibe-bot
```

```
# С локалки — синхронизируем код (БЕЗ .env и моделей NudeNet — их кладём отдельно)
rsync -avz \
  --exclude='.git' --exclude='.venv' --exclude='__pycache__' \
  --exclude='/.env' --exclude='/models' \
  ./ user@server:/opt/vibe-bot/

# На сервере
cd /opt/vibe-bot
cp .env.example .env
nano .env          # вписать prod-токен, WEBHOOK_HOST, ADMIN_TELEGRAM_IDS,
                   # YOOKASSA_PROVIDER_TOKEN, и т.п.

# Собираем образ, поднимаем БД, накатываем миграции, запускаем всё.
docker compose -f docker/docker-compose.yml build
docker compose -f docker/docker-compose.yml up -d postgres redis
docker compose -f docker/docker-compose.yml run --rm bot uv run alembic upgrade head
docker compose -f docker/docker-compose.yml up -d
```

nginx — отдельно. Образец конфига есть в `docker/nginx.conf`. Цепляем по `https://<домен>/webhook` → проксируем на `vibe-bot:8000`. Для Mini App — `https://<домен>/app/` → `vibe-api:8001` + статика.

3.3. Регулярный деплой (после первого)

```
# С локалки
rsync -avz \
  --exclude='.git' --exclude='.venv' --exclude='__pycache__' \
  --exclude='/.env' --exclude='/models' \
  ./ user@server:/opt/vibe-bot/

# На сервере
cd /opt/vibe-bot
docker compose -f docker/docker-compose.yml build bot
docker compose -f docker/docker-compose.yml run --rm bot uv run alembic upgrade head
docker compose -f docker/docker-compose.yml up -d bot api
```

⚠ **Габри** `rsync`. Используй `--exclude='/models'` (со слешем-якорем), а не `--exclude='models'`. Без слеша паттерн поймает и `app/db/models/` — модели не синхронизируются, бот падает с `AttributeError`. Это уже случалось.

⚠ **Габри** `.env`. Никогда не передавай `--delete` без явного исключения `.env` — иначе перезаträшь prod-конфиг локальным шаблоном с пустыми токенами.

4. Шпаргалка часто-используемых команд

# Зависимости / окружение	
uv sync	# установить/синхронизировать
uv add <package>	# добавить пакет
uv remove <package>	# удалить
# Тесты и линтер	
uv run pytest	# все тесты
uv run pytest tests/unit -q	# только unit
uv run pytest -k matching	# по подстроке
uv run ruff check --fix .	# авто-фикс
uv run ruff format .	# форматирование
# Миграции	

```
uv run alembic upgrade head          # накатить всё
uv run alembic downgrade -1          # откатить последнюю
uv run alembic history | head        # последние ревизии
uv run alembic revision -m "что делаем" # пустой шаблон (предпочитаем!)
# autogenerate использовать с осторожностью — см. §9 ниже.

# Docker
docker compose -f docker/docker-compose.yml logs -f bot      # логи бота
docker compose -f docker/docker-compose.yml restart bot      # рестарт
docker compose -f docker/docker-compose.yml exec bot bash    # shell внутрь
docker compose -f docker/docker-compose.yml exec postgres psql -U vibe # psql
```

Если твой shell не подцепил группу `docker` (нужен `sudo` для docker-команд), но `getent group docker` показывает твой логин — вместо релога используй `sg docker -c "docker compose ..."`.

Часть Б — Код и архитектура

5. Архитектурные правила за минуту

(Полный список — ``. Здесь — то, что чаще всего нарушают новички.)

1. **Хэндлеры — тонкие.** Только парсинг апдейта → вызов сервиса. Никаких `select(...)` в хэндлерах.
2. **Репозитории — единственное место, где SQL.** Никаких `session.execute` в сервисах/хэндлерах.
3. **Сервисы — бизнес-логика.** Не знают про `aiogram.types`, не работают с БД напрямую (вызывают репозитории).
4. **Все пользовательские строки** — в `app/texts/`. Никаких русских литералов в логике.
5. **Callback data** — только подклассы `CallbackData`, никаких `"like_42"` сырых строк.
6. **Async всюду.** Ни `time.sleep`, ни `requests`, ни синхронный `psycopg2`. Только `asyncio.sleep`, `aiohttp`, `asyncpg`.
7. **Магические числа** — либо в `app/config.py` (env-уровень), либо в таблице `app_settings` (продуктовый параметр, меняется без релиза).

6. Workflow для разраба

1. Создай ветку из main: `git checkout -b feat/название`.
2. **Сначала почитай** `` и нужный раздел — там пошаговые ТЗ по этапам.
3. Пиши код. Перед коммитом обязательно:
4. `uv run ruff check --fix .` → чисто.
5. `uv run ruff format .` → чисто.
6. `uv run pytest -q` → зелёный.
7. Если менял модели → `uv run alembic upgrade head` на свежей БД проходит.
8. Коммит-сообщение в стиле Conventional Commits: `feat(matching): add multi-vibe scoring`, `fix(profile): handle null city`, `refactor(handlers): extract render_card`.
9. PR в main → ревью → мердж.

«Зелёный pytest перед коммитом» — это hard rule, не пожелание.

7. Карта кода

```

vibe-bot/
├── app/
│   ├── main.py          # entrypoint (polling / webhook)
│   ├── config.py        # pydantic-settings (читает .env)
│   ├── logger.py        # loguru
│   ├── db/
│   │   ├── base.py      # DeclarativeBase + async engine + session factory
│   │   ├── models/      # ← BCE ORM-модели здесь. По одному файлу на сущность.
│   │   │   ├── user.py  # анкета + M2M (фаномы, интересы, вайбы, и т.п.)
│   │   │   ├── profile.py # справочники: Gender, Fandom, Interest, Vibe, ...
│   │   │   ├── dictionaries.py # Like / Dislike / Match / Block / ViewedProfile
│   │   │   ├── matching.py # PremiumSubscription, Payment
│   │   │   ├── premium.py # CreatorPost, фото, submissions
│   │   │   ├── creators.py # Complaint, ContentModerationLog, StopWord
│   │   │   ├── moderation.py # PromoPost, broadcast recipients
│   │   │   ├── promo.py
│   │   │   ├── analytics.py
│   │   │   ├── settings.py # key/value таблица app_settings
│   │   │   └── admin.py
│   │   └── repositories/ # ← ЕДИНСТВЕННОЕ место, где есть select/insert/update.
│   ├── services/        # ← Бизнес-логика. Не знает aiogram, не знает SQL.
│   │   ├── matching service.py
│   │   ├── premium service.py
│   │   ├── payment service.py
│   │   ├── moderation service.py
│   │   ├── content moderation service.py # стоп-слова + NudeNet
│   │   ├── creators service.py
│   │   ├── broadcast service.py
│   │   ├── analytics service.py
│   │   └── settings_service.py
│   └── bot/
│       ├── dispatcher.py # сборка Dispatcher + регистрация роутеров и мидлварей
│       ├── middlewares/  # user middleware (грузит User), ban, throttling
│       ├── states/       # FSM-стейты (RegistrationStates, ProfileEditStates, ...)
│       ├── keyboards/    # InlineKeyboardBuilder-обёртки + CallbackData классы
│       ├── handlers/     # ← ТОНКИЕ. Парсинг апдейта → сервис.
│       │   ├── common.py # /start, главное меню, /help
│       │   ├── registration.py # мини-FSM создания анкеты (~14 шагов)
│       │   ├── profile.py # «Моя анкета»: просмотр + редактирование полей
│       │   ├── matching.py # показ анкет, лайк/дизлайк/superlike
│       │   ├── matches.py # «Мои лайки / мэтчи»
│       │   ├── creators.py # лента + кнопка «Попасть в аллею»
│       │   ├── premium.py # экран Premium + Telegram Payments
│       │   ├── support.py
│       │   └── admin/     # ← Админский режим (отдельный subrouter)
│       └── utils/        # render profile, render_creator, vibe_picker, pagination
│   ├── api/              # FastAPI для Mini App
│   ├── scheduler/        # APScheduler: истечение Premium / постов / рассылки
│   └── texts/             # ← Все русские строки. Никаких литералов в логике.
├── alembic/versions/     # миграции; одна миграция = одно логичное изменение
├── tests/
│   ├── unit/             # без БД – pure-функции и mocked сервисы
│   └── integration/      # с реальной тестовой БД (sqlite/pg)
├── docs/                 # ТЗ, дорожная карта, этот файл
├── docker/               # Dockerfile + compose + nginx.conf
└── webapp/               # Mini App (Vite/React/TS)

```

Правило поиска: - «Куда положить новый запрос в БД?» → `app/db/repositories/` . - «Куда положить бизнес-логику?» → `app/services/` . - «Куда положить хэндлер кнопки?» → соответствующий файл в `app/bot/handlers/` . - «Где взять текст для ответа?» → `app/texts/<раздел>.py` .

8. Типичные сценарии правок (быстрые рецепты)

Добавить новое поле в анкету

1. Модель `app/db/models/profile.py` — добавь `Mapped[...]` колонку.
2. Миграция руками: `uv run alembic revision -m "add_profile_xyz"`. Опиши `op.add_column(...)` и `op.drop_column(...)` (в `downgrade!`).
3. Репозиторий `app/db/repositories/profile_repo.py` — если есть поле в `create(...)` или специфичные геттеры/сеттеры.
4. Хэндлер регистрации (`registration.py`) и/или редактирования (`profile.py`) — добавь FSM-шаг.
5. Тексты в `app/texts/registration.py` (или `profile_edit.py`).
6. Рендер карточки `app/bot/utils/render_profile.py` — если поле видно в анкете.
7. Тесты на новое поле в `tests/integration/db/test_profile_repo.py` и `unit`-тесты рендера.

Добавить новую кнопку в меню

1. Текст кнопки → `app/texts/common.py` (или отдельный модуль).
2. Билдер клавиатуры → `app/bot/keyboards/`.
3. Хэндлер → `app/bot/handlers/<раздел>.py`. Если новая секция — отдельный `Router()`, регистрация в `dispatcher.py`.
4. Не используй сырые `callback`-строки. `CallbackData` подклассом, см. `pagination.MultiSelectCb` как образец.

Добавить настройку в админке

- Параметр, который должен меняться **без редеплоя** → таблица `app_settings`. Чтение через `SettingsRepository.get_int/get_float/get_str`. Кэш в Redis (TTL 60s) — есть из коробки в `settings_service`.
- Секреты/инфраструктура → `.env` + `app/config.py`.

Запустить миграцию руками против прод-БД

```
# На сервере
cd /opt/vibe-bot
docker compose -f docker/docker-compose.yml run --rm bot uv run alembic upgrade head
```

Откат — `downgrade -1`. **Не делать `alembic downgrade base` на проде** — это снесёт всю БД.

Сделать бэкап БД

`scripts/backup_db.sh` (ежесуточный `pg_dump`). Восстановление — `scripts/restore_db.sh`.

9. Troubleshooting & грабли

Это не теория. Каждый пункт стоил минимум одного фейл-цикла. Сохранён, чтобы не повторять.

Симптом	Причина	Решение
<code>gaierror: Temporary failure in name resolution</code> при локальном запуске бота	<code>.env</code> использует <code>docker-network</code> имена (<code>postgres</code> , <code>redis</code>), бот вне	Экспортнуть <code>DATABASE_URL=...localhost:5432...</code> и <code>REDIS_URL=...localhost:6379/0</code> перед запуском

Симптом	Причина	Решение
	контейнера их не резолвит	
После rsync на сервер бот падает с <code>AttributeError: 'Profile' object has no attribute 'xxx'</code>	<code>--exclude='models'</code> без / поймал <code>app/db/models/</code>	Использовать <code>--exclude='/models'</code> (якорь к корню передачи)
Тесты падают с <code>attached to a different loop</code>	session-scoped асинхронные фикстуры + дефолтный function-loop	В <code>pyproject.toml</code> стоят <code>asyncio default fixture loop scope="session"</code> и <code>asyncio_default_test_loop_scope="session"</code> — не трогать
После прямого <code>update(Model).where(...)</code> объект из <code>session.get()</code> не обновился	identity map	<code>await db_session.refresh(obj)</code> после прямого UPDATE
<code>pydantic-settings</code> падает на <code>list[int]</code> из CSV-env	пытается парсить как JSON	Уже решено в <code>app/config.py</code> : <code>Annotated[list[int], NoDecode]</code> + <code>@field_validator(mode="before")</code> — копируй паттерн
<code>hatchling build</code> падает в Docker	требует <code>README.md</code> рядом с <code>pyproject.toml</code>	В Dockerfile копировать <code>README.md</code> вместе с <code>pyproject.toml</code> и <code>uv.lock</code> в builder-stage (уже сделано)
<code>docker compose</code> просит sudo, хотя ты в группе docker	shell не подцепил группу при логине	<code>sg docker -c "docker compose ..."</code> или relogin
<code>TelegramConflictError</code> после рестарта бота	Telegram-сервер ещё думает что старый инстанс жив	Норма ~30 сек, новый бот сам retry-нет. Не пугаться
Уведомления о жалобах не приходят	<code>ADMIN_TELEGRAM_IDS</code> пустой	Заполнить в <code>.env</code> через CSV: <code>ADMIN_TELEGRAM_IDS=123,456</code>
<code>alembic revision --autogenerate</code> подхватил «чужие» модели	autogenerate читает все импорты	Если работаешь над одной таблицей — пиши миграцию руками: <code>alembic revision -m "..."</code> без autogenerate

Остановка зависшего бота

```
# Локально (polling)
pkill -f "run bot.sh"
pkill -f "python.*app.main"

# Под docker compose
docker compose -f docker/docker-compose.yml stop bot
```

10. Безопасность и приватность (краткий чеклист перед коммитом)

- [] В `.env.example` — только заглушки. Реальный `.env` — в `.gitignore`. Никогда не коммить `BOT_TOKEN`, `YOOKASSA_PROVIDER_TOKEN`.
 - [] В логах префикс токена платежа, а не полный `provider_payment_charge_id`.
 - [] Webhook слушает только при корректном `X-Telegram-Bot-API-Secret-Token`.
 - [] Все пользовательские тексты и медиа проходят через `content_moderation_service`.
 - [] `/delete` физически удаляет анкету и соц-данные; `account` и `Premium`-статус остаются.
-

11. Куда смотреть дальше

- `` — полные правила, антипаттерны, секции по всем подсистемам.
 - **** — пошаговая дорожная карта разработки (этапы).
 - `docs/TZ_v1.1.docx` — оригинальное ТЗ. При расхождении кода и ТЗ — приоритет у ТЗ.
 - `./agents/` — специализированные субагенты для Code (`db-architect`, `aiogram-handler`, и т.д.). Если работаешь с Code, формулируй задачу — он сам выберет агента.
-

12. Когда останавливаться и спрашивать

Не угадывай — спроси или зафиксируй `BLOCKED`: в коммите, если:

- Промпт/тикет противоречит ТЗ или ``.
 - Нужно изменить структуру каталогов или стек.
 - Нужна интеграция с внешним сервисом, не описанным в ТЗ.
 - Не хватает данных от заказчика (ссылка на модератора, токен ЮKassa, и т.п.).
 - Пользовательский сценарий неоднозначен.
-

Если что-то в этом документе устарело — сначала проверь `` (там источник правды), а потом обнови этот файл.